EX09 - Dynamic Pressure with Functions - def, Parameters and return

Beginner Step-by-Step Guide

This guide follows the real file `ex09_dynamic_pressure_function.py`. It explains the problem first and then the code from the perspective of someone starting Python.

1. What we want to solve

Solve EX01 in a reusable way by placing validation and calculation inside a function.

Problem variables / elements

Name	Meaning
rho	density
speed	speed
q	dynamic pressure

Equation or logic on paper

```
q = 0.5 * rho * speed**2
```

Manual check

The physics is unchanged: rho=1.225 and speed=100 should still give 6125 Pa.

2. What we already know and what is new

Already seen:

- variables
- if/elif/else
- dynamic pressure

New in this exercise:

- def
- parameters
- arguments
- return
- None
- function call
- is not None
- reuse

3. New Python concepts

1. Define a function

Syntax: `def calculate_dynamic_pressure(rho, speed):`

def creates a function. rho and speed are local parameters that receive values when called.

2. return

Syntax: `return q`

return sends a value back to the code that made the call. It is not the same as print().

3. None

Syntax: `return None`

None represents the absence of a useful result. It is used here for invalid inputs.

4. Function call

Syntax: `calculate_dynamic_pressure(rho, speed)`

Parentheses execute the function and supply arguments to parameters.

4. Code step by step

Step 1. Define the function

```
def calculate_dynamic_pressure(rho, speed):
```

When the file runs, Python creates the function but does not calculate q yet.

Step 2. Validate inside

```
if rho < 0:
    return None
elif speed < 0:
    return None
```

The function protects itself. A return immediately ends that call.

Step 3. Calculate and return

```
q = 0.5 * rho * speed**2
return q
```

q is local. return lets its value leave the function and reach the caller.

Step 4. Call and store return

```
dynamic_pressure = calculate_dynamic_pressure(rho, speed)
```

The value from return q becomes dynamic_pressure.

Step 5. Check None

```
if dynamic_pressure is not None:
    print(...)
```

We display pressure only if the function returned a valid result.

Step 6. Reuse

```
q1 = calculate_dynamic_pressure(1.225, 100)
q2 = calculate_dynamic_pressure(1.225, 200)
```

The same logic can run with new inputs without copying internal code.

5. Execution flow

```
define function
 -> set inputs
 -> call function
 -> local validation/calculation
 -> return
 -> caller uses result
```

6. Expected result / check

The nominal call returns 6125.0 Pa. Extra calls produce 6125, 24500 and 50000 Pa.

7. Common beginner mistakes

- Confusing function definition with execution.
- Forgetting return.
- Expecting a local variable outside the function.
- Not checking None.

8. If you remember only five things

- def creates the function.
- Parameters are in the definition; arguments arrive in the call.
- return sends data back.
- Local variables live inside the function.
- Functions prevent duplicated logic.

9. Mini-practice

Change one input. Before running, write a comment predicting what should change and why. Then run the file and compare your prediction with the output.

Completion criterion

Move on when you can explain the problem, equation/logic, new Python concepts and execution flow without simply reading the code.